

COMP1786 Mobile Application Design and Development

Lieu Vinh Khoi

001391472

I. Logbook 1:

1. Basic Information

1.1 Student name	Lieu Vinh Khoi
1.2 Who did you work with? Note that for logbook exercises you are allowed to work with one other person as long as you give their name and login id and both contribute to the work.	Name: Login id:
1.3 Which Exercise is this? Tick as appropriate.	• Exercise 1 ☒ • Exercise 2 ☐ • Exercise 3 ☐
1.4 How well did you complete the exercise? Tick as appropriate.	• I tried but couldn't complete it ☐ • I did it but I feel I should have done better ☐ • I did everything that was asked ☐ • I did more than was asked for ☒
1.5 Briefly explain your answer to question 1.4 Without any explanation/justification, your scores will be deducted.	I developed a length unit converter application that allows users to convert values between a wide range of length units, from nanometers to parsecs. The application supports more units than initially required, which only specified conversions between meters, millimeters, miles, and feet. Additionally, I added input validation to ensure accurate conversions and created a clean, user-friendly interface, making the app a versatile tool for length conversions beyond the basic requirements.

2. Exercise answer

2.1 Screenshots demonstrating what you achieved

2.1.1 Application Interfaces

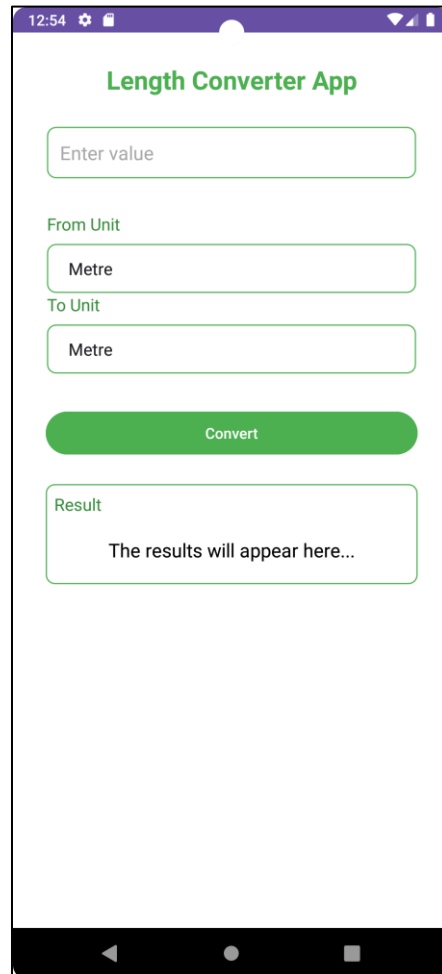
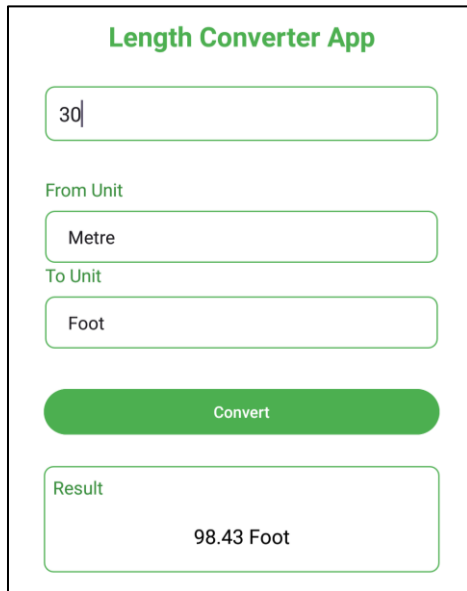


Figure 1. Main Interface of the Length Converter Application

The screenshot illustrates the main function of the Length Converter application. In Figure 1, when accessing the application, the main screen will display an interface that allows users to enter the numerical value they want to convert, then the user will select the unit to convert (for example, from Meter to Foot) and click the "Convert" button to see the results in the results section, here there is a text "The results will appear here..." to help users easily confirm the location of the results display. Showing a simple but effective interface design, optimized for easy user interaction.

2.1.2 Application features

2.1.2.1 The unit conversion function

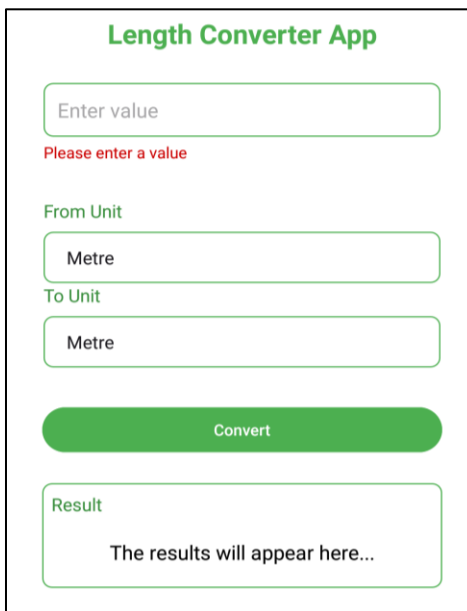


The screenshot displays the 'Length Converter App' interface. At the top, the title 'Length Converter App' is shown in green. Below it is a text input field containing the number '30'. Underneath the input field are two labels: 'From Unit' and 'To Unit'. The 'From Unit' dropdown menu is set to 'Metre', and the 'To Unit' dropdown menu is set to 'Foot'. A green 'Convert' button is positioned below these dropdowns. At the bottom, a 'Result' box displays the output '98.43 Foot'.

Figure 2. Conversion function

The screenshot above illustrates the unit conversion function. Figure 2 shows the user converting the value 30 from Meter to Foot. After pressing the "Convert" button, the result is 98.43 Foot, indicating that the conversion was successful and the displayed result is highly accurate. This feature is built to ensure accurate results and fast response, providing a smooth user experience.

2.1.2.2 Exception function when a user does not enter conversion value

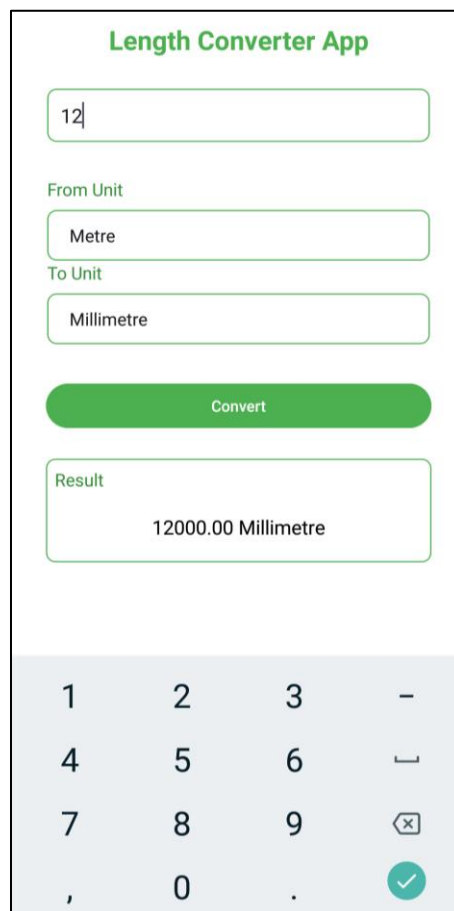


The screenshot displays the 'Length Converter App' interface with an error state. The title 'Length Converter App' is at the top in green. Below it is a text input field with the placeholder text 'Enter value'. Directly below the input field is a red error message: 'Please enter a value'. Underneath the error message are the 'From Unit' and 'To Unit' dropdown menus, both set to 'Metre'. A green 'Convert' button is located below the dropdowns. At the bottom, the 'Result' box contains the text 'The results will appear here...'.

Figure 3. Display when no value is entered

The screenshot above illustrates how the application handles exceptions when the user does not enter a value. Figure 3 shows the error message "Please enter a value!" displayed when the "Convert" button is pressed without entering data. This shows that the level of assurance in the application's system architecture is good, ensuring that no exceptions occur that lead to cracking the application, affecting the user experience, and also ensuring that the user needs to provide a valid numeric input before the conversion is performed.

2.1.2.3 Exception function prevents the user from entering alphabetic characters.



The screenshot displays the 'Length Converter App' interface. At the top, the title 'Length Converter App' is shown in green. Below it is a text input field containing the number '12'. Underneath the input field are two labels: 'From Unit' and 'To Unit'. The 'From Unit' dropdown menu is set to 'Metre', and the 'To Unit' dropdown menu is set to 'Millimetre'. A green 'Convert' button is positioned below these units. Under the 'Convert' button is a 'Result' box displaying '12000.00 Millimetre'. At the bottom of the app is a numeric keypad with buttons for digits 1 through 9, 0, a decimal point, a comma, and a green checkmark button. The keypad also includes symbols for minus, square root, and a clear/cancel function.

Figure 4. Only enter numeric characters

The screenshot demonstrates the app's numeric input feature. Figure 4 shows that users can only enter numeric characters, blocking non-numeric entries. This restriction ensures that only valid numerical data (integers or decimals) is processed for conversion, enhancing accuracy and preventing errors.

2.2 Code that you wrote

2.2.1 Directory structure

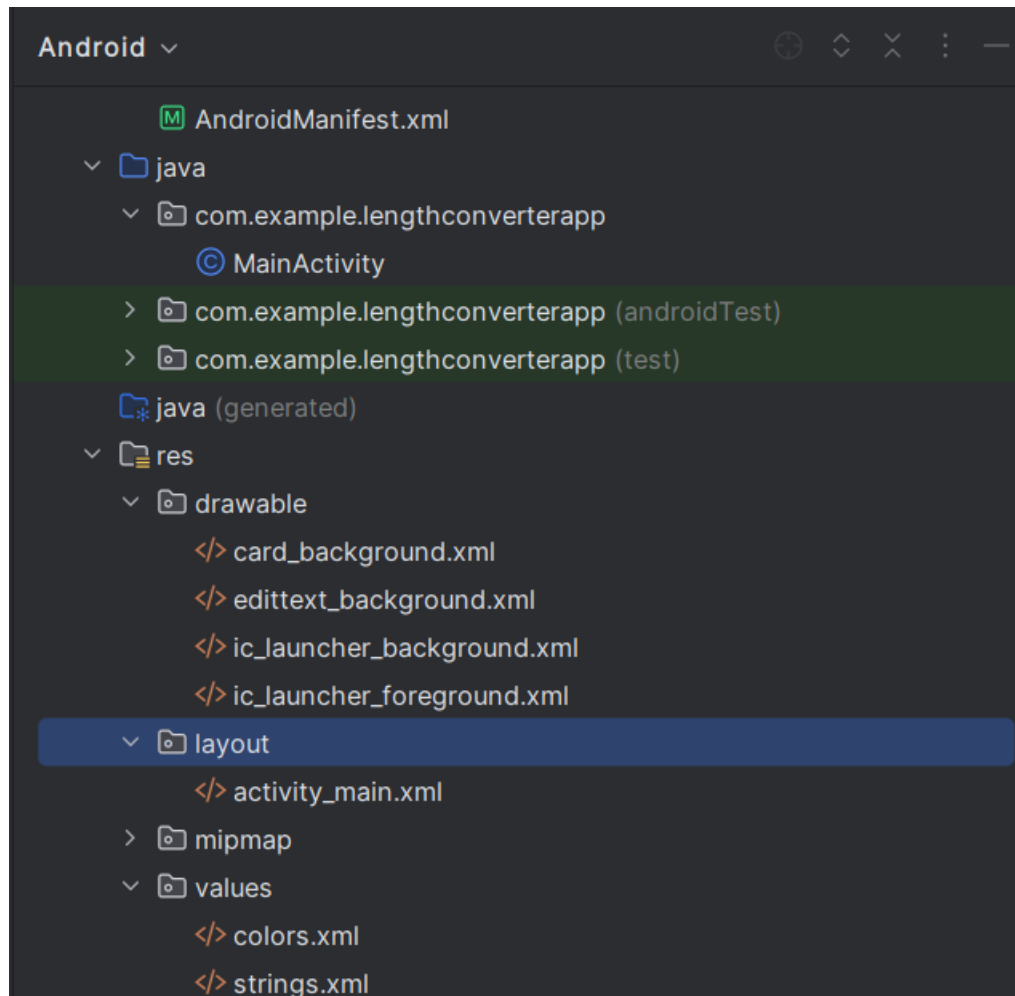


Figure 5. Directory structure of Logbook 1

The screenshot illustrates the directory tree of logbook 1. Figure 5 shows that the directory structure of logbook 1 mainly focuses on files related to Main, which means that the main interface of the application is built directly on the default main interface of the application. In addition, some drawable templates are added to create effects and an easy-to-see interface for users.

2.2.2 MainActivity.java:

```
public class MainActivity extends AppCompatActivity {
    private EditText inputValue; 2 usages
    private Spinner inputUnit, outputUnit; 3 usages
    private Button convertButton; 2 usages
    private TextView resultText, inputError; 2 usages
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        inputValue = findViewById(R.id.input_value);
        inputUnit = findViewById(R.id.input_unit);
        outputUnit = findViewById(R.id.output_unit);
        convertButton = findViewById(R.id.convert_button);
        resultText = findViewById(R.id.result_text);
        inputError = findViewById(R.id.input_error);

        ArrayAdapter<CharSequence> adapter = ArrayAdapter.
            createFromResource(context: this, R.array.length_units, android.R.layout.simple_spinner_item);
        adapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item);
        inputUnit.setAdapter(adapter);
        outputUnit.setAdapter(adapter);

        convertButton.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                String input = inputValue.getText().toString();
                if (input.isEmpty()) {
                    inputError.setVisibility(View.VISIBLE);
                    return;
                } else {
                    inputError.setVisibility(View.GONE);
                }
                double value = Double.parseDouble(input);
                String fromUnit = inputUnit.getSelectedItem().toString();
                String toUnit = outputUnit.getSelectedItem().toString();

                double result = convertLength(value, fromUnit, toUnit);
                resultText.setText(String.format("%.2f %s", result, toUnit));
            }
        });
    }
}
```

Figure 6. onCreate in MainActivity

Figure 6 demonstrates the initialization and setup of the `onCreate()` function in this Android application. Within this essential UI components such as an `EditText` for input two `Spinner` widgets for unit selection and a Button for executing the conversion are initialized. In `onCreate()` these elements are linked to their XML counterparts using `findViewById` and an `ArrayAdapter` is used to populate the `Spinner` widgets with unit options from a predefined resource. The `convertButton` is configured to listen for click events triggering input validation an error displays if the input field is empty or if valid the app proceeds with conversion. Upon clicking the function

retrieves the input value selected units and calls `convertLength` to calculate and display the result in `resultText` ensuring a smooth validated conversion experience.

```
private static final double NANOMETRE_TO_METRE = 1e-9; 1 usage
private static final double MICROMETRE_TO_METRE = 1e-6; 1 usage
private static final double MILLIMETRE_TO_METRE = 1e-3; 1 usage
private static final double CENTIMETRE_TO_METRE = 1e-2; 1 usage
private static final double DECIMETRE_TO_METRE = 1e-1; 1 usage
private static final double METRE_TO_METRE = 1; // Base unit 1 usage
private static final double KILOMETRE_TO_METRE = 1e3; 1 usage
private static final double MILE_TO_METRE = 1609.34; 1 usage
private static final double NAUTICAL_MILE_TO_METRE = 1852; 1 usage
private static final double FOOT_TO_METRE = 0.3048; 1 usage
private static final double INCH_TO_METRE = 0.0254; 1 usage
private static final double YARD_TO_METRE = 0.9144; 1 usage
private static final double LIGHT_YEAR_TO_METRE = 9.461e15; 1 usage
private static final double ASTRONOMICAL_UNIT_TO_METRE = 1.496e11; 1 usage
private static final double PARSEC_TO_METRE = 3.086e16; 1 usage
```

Figure 7. Conversion factors relative to 1 meter

```
private double convertLength(double value, String fromUnit, String toUnit) {
    double metreValue = value * getConversionFactorToMetre(fromUnit);
    return metreValue / getConversionFactorToMetre(toUnit);
}
```

Figure 8. convertLength() in MainActivity

```
private double getConversionFactorToMetre(String unit) { 2 usages
    switch (unit) {
        case "Nanometre":
            return NANOMETRE_TO_METRE;
        case "Micrometre":
            return MICROMETRE_TO_METRE;
        case "Millimetre":
            return MILLIMETRE_TO_METRE;
        case "Centimetre":
            return CENTIMETRE_TO_METRE;
        case "Decimetre":
            return DECIMETRE_TO_METRE;
        case "Metre":
            return METRE_TO_METRE;
        case "Kilometre":
            return KILOMETRE_TO_METRE;
        case "Mile":
            return MILE_TO_METRE;
        case "Nautical Mile":
            return NAUTICAL_MILE_TO_METRE;
        case "Foot":
            return FOOT_TO_METRE;
        case "Inch":
            return INCH_TO_METRE;
        case "Yard":
            return YARD_TO_METRE;
        case "Light Year":
            return LIGHT_YEAR_TO_METRE;
        case "Astronomical Unit":
            return ASTRONOMICAL_UNIT_TO_METRE;
        case "Parsec":
            return PARSEC_TO_METRE;
        default:
            throw new IllegalArgumentException("Unsupported unit: " + unit);
    }
}
```

Figure 9. getConversionFactorToMetre() in MainActivity

Figures 7, 8, and 9 illustrate the optimized `convertLength` function for unit conversions. In Figure 7, each unit's conversion factor to meters is defined as a constant, centralizing conversion values for easy maintenance. Figure 8 shows the `convertLength` method, which converts the input value to meters using the factor from `getConversionFactorToMetre` based on `fromUnit`, then divides by the target unit's factor to get the final result. Figure 9 shows `getConversionFactorToMetre`, a method that retrieves the conversion factor for each unit through a `switch` statement, ensuring organized and flexible conversions across various units.

2.2.3 strings.xml:

```
<resources>
    <string name="app_name">Length Converter</string>
    <string-array name="length_units">
        <item>Nanometre</item>
        <item>Micrometre</item>
        <item>Millimetre</item>
        <item>Centimetre</item>
        <item>Decimetre</item>
        <item>Metre</item>
        <item>Kilometre</item>
        <item>Mile</item>
        <item>Nautical Mile</item>
        <item>Foot</item>
        <item>Inch</item>
        <item>Yard</item>
        <item>Light Year</item>
        <item>Astronomical Unit</item>
        <item>Parsec</item>
    </string-array>

    <string name="hint_enter_value">Enter value</string>
    <string name="convert_button_text">Convert</string>
    <string name="result_text">Result</string>
</resources>
```

Figure 10. strings.xml

The `strings.xml` file organizes all text resources for the app, like the app name, unit options, input hints, button labels, and result text. This setup allows easy modification of display text, as changes in `strings.xml` automatically update text across the app. By defining elements like `app\_name` and `convert\_button\_text`, the file ensures consistent text management and a streamlined localization process.

2.2.4 themes.xml and activity_main.xml:

```
<resources xmlns:tools="http://schemas.android.com/tools">
    <!-- Base application theme. -->
    <style name="Base.Theme.LengthConverterApp" parent="Theme.Material3.DayNight.NoActionBar">
        <!-- Customize your light theme here. -->
        <!-- <item name="colorPrimary">@color/my_light_primary</item> -->
    </style>

    <style name="Theme.LengthConverterApp" parent="Base.Theme.LengthConverterApp" />

    <style name="CustomSpinnerStyle">
        <item name="android:background">@drawable/edittext_background</item>
        <item name="android:padding">12dp</item>
        <item name="android:textColor">@color/colorPrimaryDark</item>
    </style>
</resources>
```

Figure 11. themes.xml

```
<TextView
    android:id="@+id/input_label"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="From Unit"
    android:textSize="16sp"
    android:textColor="@color/colorPrimaryDark"
    android:layout_marginBottom="8dp" />

<Spinner
    android:id="@+id/input_unit"
    style="@style/CustomSpinnerStyle"
    android:layout_width="match_parent"
    android:layout_height="wrap_content" />

<TextView
    android:id="@+id/output_label"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="To Unit"
    android:textSize="16sp"
    android:textColor="@color/colorPrimaryDark"
    android:layout_marginBottom="8dp" />

<Spinner
    android:id="@+id/output_unit"
    style="@style/CustomSpinnerStyle"
    android:layout_width="match_parent"
    android:layout_height="wrap_content" />
```

Figure 12. Select conversion unit in activity_main.xml

In this setup, the custom style `CustomSpinnerStyle` is defined in `themes.xml` to standardize the appearance of `Spinner` widgets. This style sets a background drawable, padding, and text color, ensuring a consistent look. In `activity\_main.xml`, the style is applied to both `input\_unit` and `output\_unit` spinners using `style="@style/CustomSpinnerStyle"`. This approach allows any visual updates to the style in `themes.xml` to reflect automatically across all spinners using it, simplifying maintenance and ensuring uniformity.

II. Logbook 2:

1. Basic Information

1.1 Student name	Lieu Vinh Khoi
1.2 Who did you work with? Note that for logbook exercises you are allowed to work with one other person as long as you give their name and login id and both contribute to the work.	Name: Login id:
1.3 Which Exercise is this? Tick as appropriate.	• Exercise 1 ☐ • Exercise 2 ☒ • Exercise 3 ☐
1.4 How well did you complete the exercise? Tick as appropriate.	• I tried but couldn't complete it ☐ • I did it but I feel I should have done better ☐ • I did everything that was asked ☐ • I did more than was asked for ☒
1.5 Briefly explain your answer to question 1.4 Without any explanation/justification, your scores will be deducted.	I built a task management application that allows users to add, edit, and delete tasks. Each task is associated with a name and includes additional fields such as description, due date, time, and a label for categorization. Users can mark tasks as completed, which visually updates the task item. This app goes beyond the basic requirements by providing additional features like date and time selection, categorization, and completion status for each task, offering a more comprehensive and organized to-do list experience.

2. Exercise answer

2.1 Screenshots demonstrating what you achieved

2.1.1 Application Interfaces

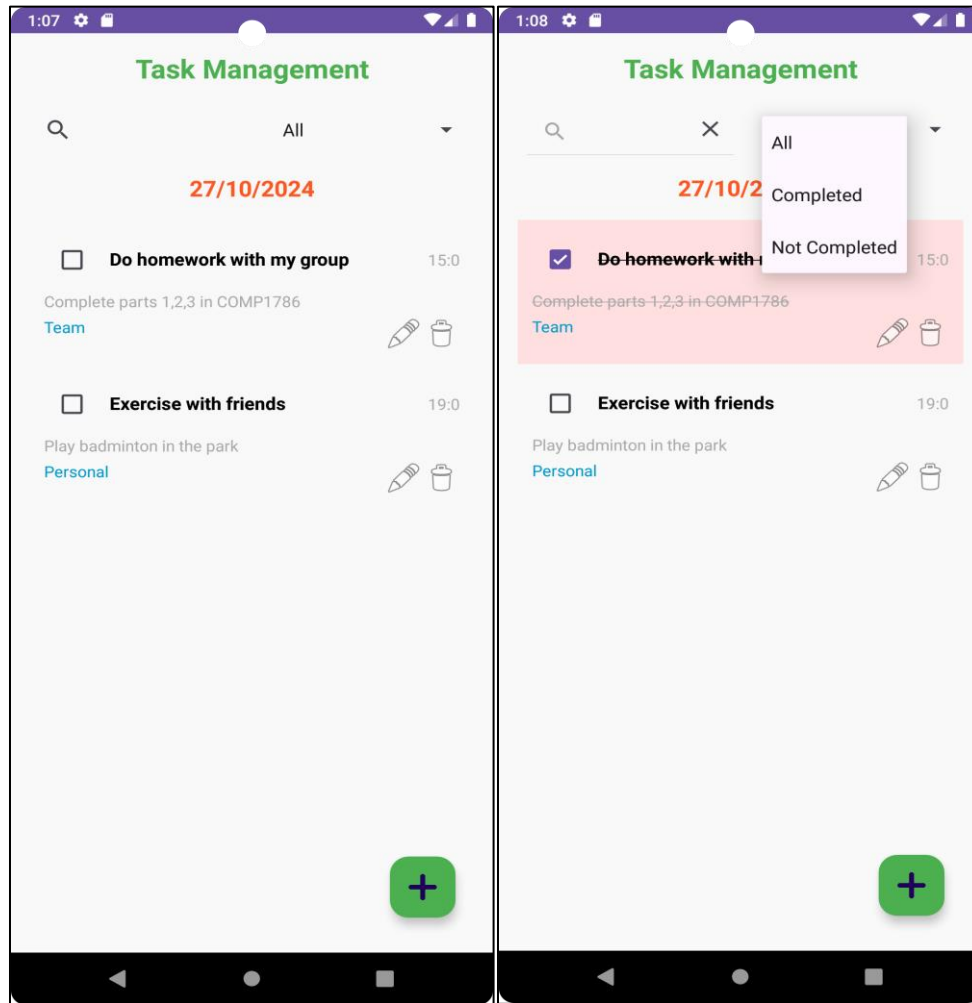


Figure 13. Main Interface of the Task Management Application

Figure 13 illustrates the main interface of the Task Manager application, designed for clear and organized task management. The toolbar at the top includes a search bar and filter for viewing tasks by completion status. Below, a date display section allows users to filter tasks by selected dates. The task list shows each task with a checkbox to mark completion; completed tasks are highlighted with a different background color and a strikethrough on the task name for easy identification. Users can also edit or delete tasks using icons next to each item. A floating action

button in the lower right enables quick task addition, enhancing management efficiency and providing a user-friendly experience.

2.1.2 Application features

2.1.2.1 Add a new task

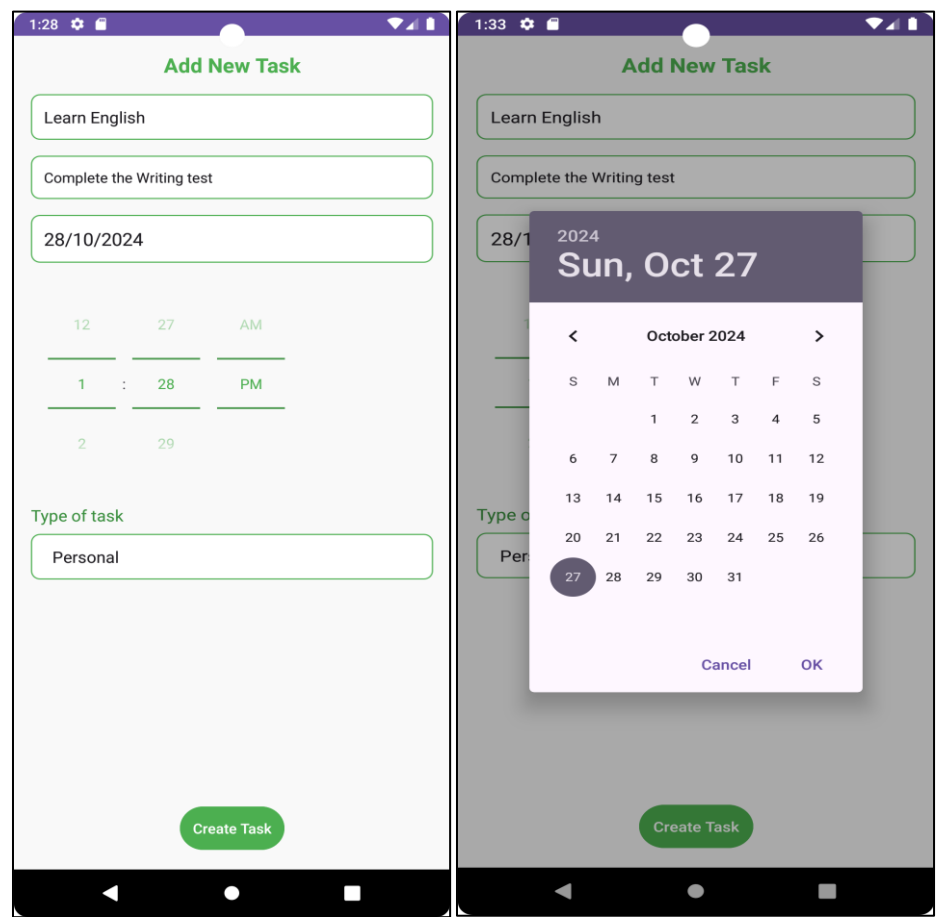
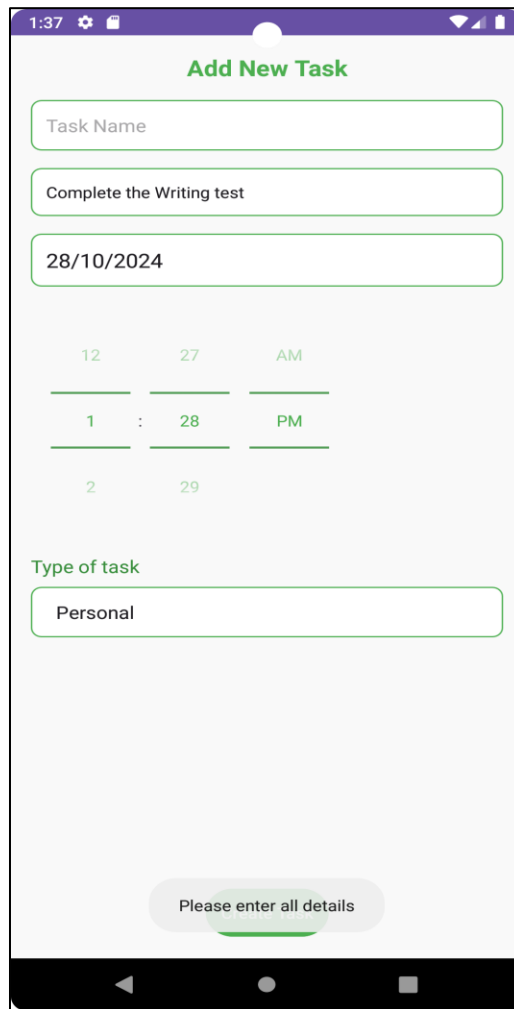


Figure 14. Add new task interface

Figure 14 shows the "Add New Task" interface of the Task Manager application. At the top, a header labeled "Add New Task" sets the purpose of the screen. Below the header, users can fill in task details with three input fields: one for the task name, one for a description, and one to select the task date. A time picker allows users to set the specific time for the task, using an intuitive AM/PM format. Further down, a dropdown menu labeled "Type of Task" lets users categorize the task (e.g., Personal, Work, Team). At the bottom of the screen, a green "Create Task" button enables users to save the task, completing the input process. This layout ensures a straightforward, step-by-step flow for adding new tasks efficiently.

2.1.2.2 Catch the exception of not enough input fields



The screenshot displays a mobile application interface titled "Add New Task". The form contains the following elements:

- Task Name:** A text input field with the placeholder text "Task Name".
- Task Description:** A text input field containing the text "Complete the Writing test".
- Due Date:** A date input field containing "28/10/2024".
- Time Picker:** A time selection interface showing the time "12:28 AM". Below this, the numbers "1", "28", and "PM" are visible, along with "2" and "29" below them.
- Type of task:** A dropdown menu currently showing "Personal".
- Error Message:** A light green rounded rectangle at the bottom of the form contains the text "Please enter all details".

The interface is set against a light gray background with a purple header bar at the top. The status bar at the very top shows the time "1:37" and various system icons.

Figure 15. Show a message not enough input fields are filled

Figure 15 shows the "Add New Task" interface with an error message displayed at the bottom. The message "Please enter all details" indicates that the user attempted to submit the form without filling in all required fields. This error prompt ensures users provide all necessary details to successfully create a task.

2.1.2.3 Catch past date selection exception

The screenshot displays a mobile application interface titled "Add New Task". It features three input fields: "Learn English", "Complete the Writing test", and "25/10/2024". Below these is a time selection interface with two rows of options: the first row shows "12", "27", and "AM"; the second row shows "1", "28", and "PM". A third row shows "2" and "29". Below the time selection is a "Type of task" section with a dropdown menu set to "Personal". At the bottom, a green error message box states "Selected date and time are in the past!". The interface is set against a light gray background with a purple header bar.

Figure 16. Show a message to select a date and time in the past

Figure 16 shows the "Add New Task" interface with an error message displayed at the bottom, stating, "Selected date and time are in the past!" This notification alerts the user that the chosen date and time are not valid for scheduling a future task. This feedback prevents users from accidentally setting past deadlines.

2.1.2.4 Update a task

The figure displays three sequential screenshots of a mobile application's 'Update Task' screen. Each screen has a purple header with the title 'Update Task' in green. The interface includes several input fields: 'Task Name', 'Description', 'Date', 'Time', and 'Type of task'. The 'Date' and 'Time' fields are implemented as digital clocks with dropdown menus for hours, minutes, and periods (AM/PM). A green 'Update Task' button is located at the bottom of each screen.

- Left Screenshot (1:55):** Shows the task being updated. The 'Task Name' field contains 'Do homework with my group', the 'Description' contains 'Complete parts 1,2,3 in COMP1786', the 'Date' is '27/10/2024', and the 'Time' is '2:59 AM'. The 'Type of task' is 'Work'. The 'Update Task' button is visible at the bottom.
- Middle Screenshot (1:56):** Shows the same task details, but the 'Task Name' field is empty. A grey error message 'Please enter all details' is displayed at the bottom.
- Right Screenshot (1:58):** Shows the same task details, but the 'Date' is '25/10/2024' and the 'Time' is '2:00 PM'. A grey error message 'Selected date and time are in the past!' is displayed at the bottom.

Figure 17. Update a task interface

Figure 17 illustrates the Update Task interface, where the user can edit the details of a task. This screen is similar to the Add Task layout, including fields for the task name, description, date, time, and task type. The title reads "Update Task" and the "Update Task" button is displayed at the bottom. The available information for the task is displayed corresponding to the fields in the interface. When the user leaves required fields blank or selects a date/time in the past, an error message appears, such as "Please enter all details" or "The selected date and time are in the past!" The interface provides visual feedback to handle errors and ensure valid input for updating the task.

2.1.2.5 Delete a task

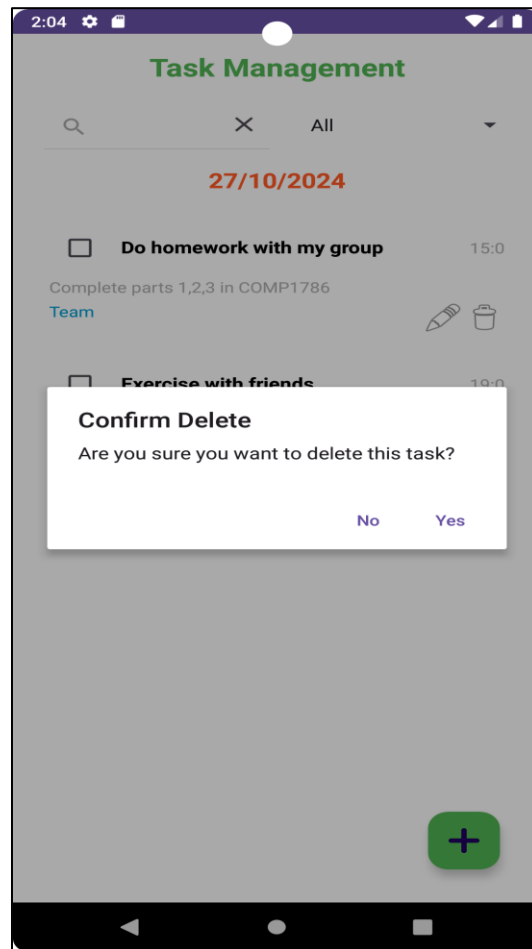


Figure 18. Display a popup confirming the task deletion

Figure 18 shows the Confirm Delete dialog in the Task Management application. When a user selects the delete icon for a task, this confirmation popup appears, asking, "Are you sure you want to delete this task?" with options for "No" to cancel and "Yes" to proceed with the deletion. This confirmation step prevents accidental deletion, adding a layer of security to task management.

2.1.2.6 Search for tasks

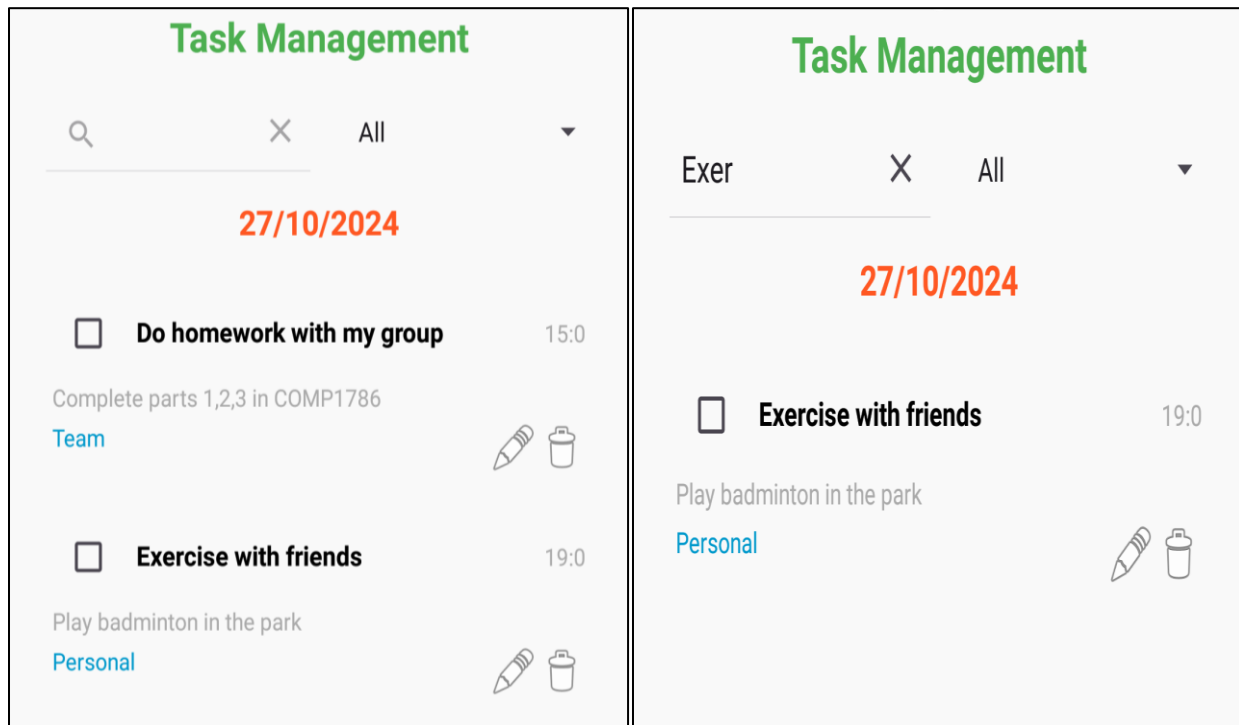


Figure 19. Search for tasks

Figure 19 displays the search functionality in the Task Management application. When a user types a keyword (e.g., "Exer") in the search bar, the system filters and shows only tasks with names matching the entered text. Here, only the task "Exercise with friends" appears, as it matches the search criteria. This feature helps users quickly locate specific tasks from their list.

2.1.2.7 Filter tasks by completion

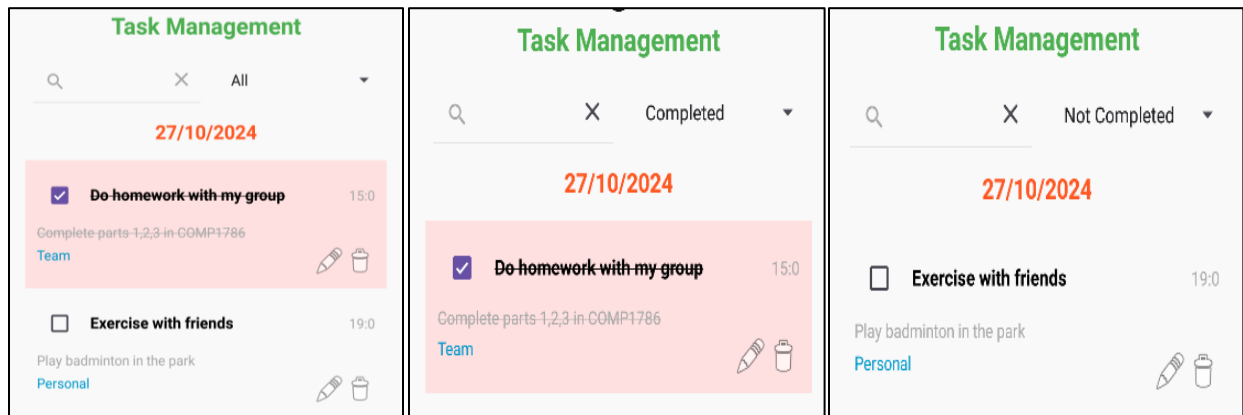


Figure 20. Filter tasks by completion

In Figure 20, the Task Management application's interface demonstrates the filtering feature based on task completion status. When users select "Completed" in the filter, only tasks marked as finished are displayed, with a distinct background color and strikethrough text for easy identification. Conversely, selecting "Not Completed" shows only pending tasks. This filter system allows users to quickly view tasks based on their completion state, enhancing task organization and accessibility.

2.2 Code that you wrote

2.2.1 Directory structure



Figure 21. Directory structure of Logbook 2

The logbook2 project structure organizes main functionality into AddTaskActivity and MainActivity for task management, with Task as the data model and TaskAdapter for displaying tasks. In res, the drawable folder holds icons and custom backgrounds, layout contains XML files for the UI, and values store colors, strings, and styles. This setup ensures organized, reusable resources for consistent UI and functionality across the app.

2.2.2 MainActivity.java:

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);

    RecyclerView taskRecyclerView = findViewById(R.id.taskRecyclerView);
    FloatingActionButton addTaskButton = findViewById(R.id.addTaskButton);
    dateHeader = findViewById(R.id.dateHeader);
    emptyTextView = findViewById(R.id.emptyTextView);
    SearchView searchView = findViewById(R.id.searchView);

    Spinner filterSpinner = findViewById(R.id.filterSpinner);

    taskList = new ArrayList<>();
    allTasks = new ArrayList<>();
    taskAdapter = new TaskAdapter(taskList, context: this, this::deleteTask);

    taskRecyclerView.setLayoutManager(new LinearLayoutManager(context: this));
    taskRecyclerView.setAdapter(taskAdapter);

    Calendar calendar = Calendar.getInstance();
    SimpleDateFormat dateFormat = new SimpleDateFormat(pattern: "dd/MM/yyyy", Locale.getDefault());
    selectedDate = dateFormat.format(calendar.getTime());
    dateHeader.setText(selectedDate);
    filterTasks();

    addTaskButton.setOnClickListener(v -> {
        Intent intent = new Intent(packageContext: MainActivity.this, AddTaskActivity.class);
        startActivityForResult(intent, REQUEST_CODE_ADD_TASK);
    });

    dateHeader.setOnClickListener(v -> showDatePickerDialog());
}
```

Figure 22. onCreate() function in MainActivity

```

searchView.setOnQueryTextListener(new SearchView.OnQueryTextListener() {
    @Override 1 usage
    public boolean onQueryTextSubmit(String query) { return false; }

    @Override 1 usage
    public boolean onQueryTextChange(String newText) {
        searchQuery = newText.toLowerCase();
        filterTasks();
        return true;
    }
});

ArrayAdapter<CharSequence> adapter = ArrayAdapter.createFromResource(context, this,
    R.array.filter_options, android.R.layout.simple_spinner_item);
adapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item);
filterSpinner.setAdapter(adapter);

filterSpinner.setOnItemSelectedListener(new AdapterView.OnItemSelectedListener() {
    @Override no usages
    public void onItemSelected(AdapterView<?> parent, View view, int position, long id) {
        switch (position) {
            case 1:
                filterCompleted = true;
                filterNotCompleted = false;
                break;
            case 2:
                filterCompleted = false;
                filterNotCompleted = true;
                break;
            default:
                filterCompleted = false;
                filterNotCompleted = false;
                break;
        }
        filterTasks();
    }

    @Override no usages
    public void onNothingSelected(AdapterView<?> parent) {
    }
});

```

Figure 23. onCreate() function in MainActivity

The onCreate() method in MainActivity initializes the main UI components and functionality of the app. It sets up the RecyclerView for displaying tasks, the FloatingActionButton for adding new tasks, and the dateHeader to show and filter tasks by date. It also configures a SearchView for filtering tasks based on the search query and a Spinner to filter tasks by completion status. Additionally, the method sets up listeners for date selection, adding tasks, and filtering options, enabling a streamlined user experience for managing tasks.

```

private void showDatePickerDialog() { 1 usage
    final Calendar calendar = Calendar.getInstance();
    int year = calendar.get(Calendar.YEAR);
    int month = calendar.get(Calendar.MONTH);
    int day = calendar.get(Calendar.DAY_OF_MONTH);

    DatePickerDialog datePickerDialog = new DatePickerDialog(context, this, (view, year1, month1, dayOfMonth) -> {
        selectedDate = dayOfMonth + "/" + (month1 + 1) + "/" + year1;
        dateHeader.setText(selectedDate);
        filterTasks();
    }, year, month, day);
    datePickerDialog.show();
}

private void filterTasks() { 5 usages
    taskList.clear();
    for (Task task : allTasks) {
        boolean matchesSearch = task.getName().toLowerCase().contains(searchQuery);
        boolean matchesFilter = (!filterCompleted && !filterNotCompleted) ||
            (filterCompleted && task.isCompleted()) ||
            (filterNotCompleted && !task.isCompleted());

        if (matchesSearch && matchesFilter && task.getDate().equals(selectedDate)) {
            taskList.add(task);
        }
    }
    taskAdapter.notifyDataSetChanged();
    emptyTextView.setVisibility(taskList.isEmpty() ? View.VISIBLE : View.GONE);
}

private void deleteTask(int position) { 1 usage
    Task task = taskList.get(position);
    allTasks.remove(task);
    taskList.remove(position);
    taskAdapter.notifyItemRemoved(position);
    taskAdapter.notifyItemRangeChanged(position, taskList.size());

    if (taskList.isEmpty()) {
        emptyTextView.setVisibility(View.VISIBLE);
    }
}
}

```

Figure 24. Others function in MainActivity

The three methods in MainActivity manage date selection, task filtering, and task deletion. The `showDatePickerDialog()` method displays a date picker dialog, allowing users to select a date, which updates the displayed date and filters tasks accordingly. The `filterTasks()` method filters tasks based on the search query, selected date, and completion status, updating the displayed list and showing an empty message if no tasks match. Finally, the `deleteTask()` method removes a selected task from both `taskList` and `allTasks`, updating the UI and showing a message if no tasks remain. These methods streamline date selection, filtering, and task removal.

```

@Override
protected void onActivityResult(int requestCode, int resultCode, @Nullable Intent data) {
    super.onActivityResult(requestCode, resultCode, data);

    if ((requestCode == REQUEST_CODE_ADD_TASK || requestCode == REQUEST_CODE_EDIT_TASK) && resultCode == RESULT_OK && data != null) {
        String taskName = data.getStringExtra("taskName");
        String taskDescription = data.getStringExtra("taskDescription");
        String taskTime = data.getStringExtra("taskTime");
        String taskLabel = data.getStringExtra("taskLabel");
        String taskDate = data.getStringExtra("taskDate");
        boolean isEditMode = data.hasExtra("taskPosition");

        if (isEditMode) {
            int position = data.getIntExtra("taskPosition", -1);
            Task task = allTasks.get(position);
            task.setName(taskName);
            task.setDescription(taskDescription);
            task.setTime(taskTime);
            task.setLabel(taskLabel);
            task.setDate(taskDate);
            taskAdapter.notifyItemChanged(position);
        } else {
            Task newTask = new Task(taskName, taskDescription, taskTime, taskLabel, taskDate);
            allTasks.add(newTask);
        }

        filterTasks();
    }
}

```

Figure 25. Other function in MainActivity

The `onActivityResult()` method in `MainActivity` handles the result returned from adding or editing a task. When a task is added or edited and saved, this method retrieves task details from the Intent data, including task name, description, time, label, and date. If in edit mode, it updates the specific task's details in `allTasks` and refreshes the `RecyclerView`. If adding a new task, it creates a new task object and adds it to `allTasks`. Finally, it calls `filterTasks()` to apply any filters and refresh the displayed task list, ensuring the main interface reflects the latest task data.

2.2.3 AddTaskActivity.java:

```

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_add_task);

    taskNameInput = findViewById(R.id.taskNameInput);
    taskDescriptionInput = findViewById(R.id.taskDescriptionInput);
    taskDateInput = findViewById(R.id.taskDateInput);
    taskTimePicker = findViewById(R.id.taskTimePicker);
    labelSpinner = findViewById(R.id.labelSpinner);
    createTaskButton = findViewById(R.id.createTaskButton);
    headerText = findViewById(R.id.headerText);

    isEditMode = getIntent().getBooleanExtra( name: "isEditMode", defaultValue: false);
    if (isEditMode) {
        taskPosition = getIntent().getIntExtra( name: "taskPosition", defaultValue: -1);
        taskNameInput.setText(getIntent().getStringExtra( name: "taskName"));
        taskDescriptionInput.setText(getIntent().getStringExtra( name: "taskDescription"));
        String[] timeParts = getIntent().getStringExtra( name: "taskTime").split( regex: ":" );
        taskTimePicker.setHour(Integer.parseInt(timeParts[0]));
        taskTimePicker.setMinute(Integer.parseInt(timeParts[1]));
        taskDateInput.setText(getIntent().getStringExtra( name: "taskDate"));
        createTaskButton.setText("Update Task");
        headerText.setText("Update Task");
    } else {
        headerText.setText("Add New Task");
    }

    taskDateInput.setOnClickListener(v -> showDatePickerDialog());
}

```

Figure 26. onCreate() function in AddTaskActivity

```

createTaskButton.setOnClickListener(v -> {
    String name = taskNameInput.getText().toString();
    String description = taskDescriptionInput.getText().toString();
    String time = taskTimePicker.getHour() + ":" + taskTimePicker.getMinute();
    String label = labelSpinner.getSelectedItem().toString();
    String date = taskDateInput.getText().toString();

    if (isDateTimeInPast()) {
        Toast.makeText( context: this, text: "Selected date and time are in the past!", Toast.LENGTH_SHORT).show();
        return;
    }

    if (!name.isEmpty() && !date.isEmpty()) {
        Intent resultIntent = new Intent();
        resultIntent.putExtra( name: "taskName", name);
        resultIntent.putExtra( name: "taskDescription", description);
        resultIntent.putExtra( name: "taskTime", time);
        resultIntent.putExtra( name: "taskLabel", label);
        resultIntent.putExtra( name: "taskDate", date);

        if (isEditMode) {
            resultIntent.putExtra( name: "taskPosition", taskPosition);
        }

        setResult(RESULT_OK, resultIntent);
        finish();
    } else {
        Toast.makeText( context: this, text: "Please enter all details", Toast.LENGTH_SHORT).show();
    }
});
}

```

Figure 27. onCreate() function in AddTaskActivity

The `onCreate()` method in `AddTaskActivity` sets up the interface for adding or updating a task. It initializes input fields, a time picker, and labels for the task. If in edit mode, it populates the fields with the task's current information and changes the header and action button to "Update Task." Users can select a date, enter task details, and, upon pressing the create or update button, the app checks if all fields are filled and ensures the date/time is not in the past. If valid, the task details are sent back to `MainActivity` for further processing.

```
private boolean isDateTimeInPast() { 1 usage
    Calendar currentCalendar = Calendar.getInstance();

    Calendar selectedCalendar = Calendar.getInstance();
    String[] dateParts = taskDateInput.getText().toString().split("regex: "/"");

    if (dateParts.length < 3) return false;

    int day = Integer.parseInt(dateParts[0]);
    int month = Integer.parseInt(dateParts[1]) - 1;
    int year = Integer.parseInt(dateParts[2]);

    int hour = taskTimePicker.getHour();
    int minute = taskTimePicker.getMinute();

    selectedCalendar.set(year, month, day, hour, minute);

    return selectedCalendar.before(currentCalendar);
}

private void showDatePickerDialog() {
    final Calendar calendar = Calendar.getInstance();
    int year = calendar.get(Calendar.YEAR);
    int month = calendar.get(Calendar.MONTH);
    int day = calendar.get(Calendar.DAY_OF_MONTH);

    DatePickerDialog datePickerDialog = new DatePickerDialog(context: this, (view, year1, month1, dayOfMonth) -> {
        String selectedDate = dayOfMonth + "/" + (month1 + 1) + "/" + year1;
        taskDateInput.setText(selectedDate);
    }, year, month, day);
    datePickerDialog.show();
}
```

Figure 28. Others function in `AddTaskActivity`

The `isDateTimeInPast()` method checks if the selected date and time are in the past compared to the current date and time. It parses the date and time from user input, sets it on a calendar instance, and compares it to the current date. The `showDatePickerDialog()` method displays a date picker dialog to allow users to select a date. When a date is selected, it updates the date input field with the chosen date in the specified format. These methods ensure users can only select valid, future dates for tasks.

2.2.4 Task.java:

```
public class Task { 9 usages
    private String name; 3 usages
    private String description; 3 usages
    private String time; 3 usages
    private String label; 3 usages
    private String date; // Thêm trường date để lưu ngày 3 usages
    private boolean completed; 3 usages

    public Task(String name, String description, String time, String label, String date) { 15 usages
        this.name = name;
        this.description = description;
        this.time = time;
        this.label = label;
        this.date = date;
        this.completed = false;
    }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public String getDescription() { return description; } 2 usages
    public void setDescription(String description) { this.description = description; } 1 usage

    public String getTime() { return time; } 2 usages
    public void setTime(String time) { this.time = time; } 1 usage

    public String getLabel() { return label; } 2 usages
    public void setLabel(String label) { this.label = label; } 1 usage

    public String getDate() { return date; } 2 usages
    public void setDate(String date) { this.date = date; } 1 usage

    public boolean isCompleted() { return completed; } 4 usages

    public void setCompleted(boolean completed) { this.completed = completed; }
}
```

Figure 29. Fields in the Task class

The Task class represents a task with attributes like `name`, `description`, `time`, `label`, `date`, and a `completed` status. It provides getter and setter methods for each attribute, allowing access and modification of the task's details. The `isCompleted()` and `setCompleted()` methods handle the task's completion status, enabling functionality to mark tasks as completed or pending. This structure makes managing task properties straightforward within the application.

2.2.5 TaskAdapter.java:

```
public interface DeleteTaskCallback { 2 usages
    void deleteTask(int position); 1 usage
}

public TaskAdapter(ArrayList<Task> taskList, Context context, DeleteTaskCallback deleteTaskCallback) { 1 usage
    this.taskList = taskList;
    this.context = context;
    this.deleteTaskCallback = deleteTaskCallback;
}

@NonNull
@Override
public TaskViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {
    View view = LayoutInflater.from(parent.getContext()).inflate(R.layout.task_item, parent, attachToRoot: false);
    return new TaskViewHolder(view);
}
```

Figure 30. Others function in TaskAdapter

In `TaskAdapter.java`, the `DeleteTaskCallback` function defines a `deleteTask` method to handle task deletion by position. The `TaskAdapter` constructor initializes the adapter with a list of tasks, the context, and a callback for deleting tasks. The `onCreateViewHolder` method inflates the `task\_item` layout for each task, creating a `TaskViewHolder` to manage item views within the RecyclerView. This setup facilitates displaying and managing tasks in the task list efficiently.

```
@Override
public void onBindViewHolder(@NonNull TaskViewHolder holder, int position) {
    Task task = taskList.get(position);

    holder.taskName.setText(task.getName());
    holder.taskDescription.setText(task.getDescription());
    holder.taskTime.setText(task.getTime());
    holder.taskLabel.setText(task.getLabel());
    holder.taskCheckBox.setChecked(task.isCompleted());

    updateTaskUI(holder, task.isCompleted());

    holder.taskCheckBox.setOnCheckedChangeListener((buttonView, isChecked) -> {
        task.setCompleted(isChecked);
        updateTaskUI(holder, isChecked);
    });

    holder.editButton.setOnClickListener(v -> {
        Intent intent = new Intent(context, AddTaskActivity.class);
        intent.putExtra( name: "isEditMode", value: true);
        intent.putExtra( name: "taskPosition", position);
        intent.putExtra( name: "taskName", task.getName());
        intent.putExtra( name: "taskDescription", task.getDescription());
        intent.putExtra( name: "taskTime", task.getTime());
        intent.putExtra( name: "taskLabel", task.getLabel());
        intent.putExtra( name: "taskDate", task.getDate());
        ((MainActivity) context).startActivityForResult(intent, MainActivity.REQUEST_CODE_EDIT_TASK);
    });

    holder.deleteButton.setOnClickListener(v -> new AlertDialog.Builder(context)
        .setTitle("Confirm Delete")
        .setMessage("Are you sure you want to delete this task?")
        .setPositiveButton( text: "Yes", (dialog, which) -> deleteTaskCallback.deleteTask(position))
        .setNegativeButton( text: "No", listener: null)
        .show());
}
```

Figure 31. onBindViewHolder() function in TaskAdapter

The `onBindViewHolder()` method in `TaskAdapter.java` binds task data to the UI elements within each item view in the `RecyclerView`. It displays task details, updates the UI if a task is marked as completed, and allows users to toggle task completion status. The `editButton` enables users to edit a task, passing its details to `AddTaskActivity`. The `deleteButton` triggers a confirmation dialog before deleting the task through a callback. This method ensures each task item is interactive and visually responsive to user actions.

```
private void updateTaskUI(TaskViewHolder holder, boolean isCompleted) { 2 usages
    if (isCompleted) {
        holder.taskName.setPaintFlags(holder.taskName.getPaintFlags() | Paint.STRIKE_THRU_TEXT_FLAG);
        holder.taskDescription.setPaintFlags(holder.taskDescription.getPaintFlags() | Paint.STRIKE_THRU_TEXT_FLAG);
        holder.taskItemContainer.setBackgroundColor(Color.parseColor("colorString: #FFE0E0"));
    } else {
        holder.taskName.setPaintFlags(holder.taskName.getPaintFlags() & ~Paint.STRIKE_THRU_TEXT_FLAG);
        holder.taskDescription.setPaintFlags(holder.taskDescription.getPaintFlags() & ~Paint.STRIKE_THRU_TEXT_FLAG);
        holder.taskItemContainer.setBackgroundColor(Color.TRANSPARENT);
    }
}

@Override
public int getItemCount() { return taskList.size(); }

public static class TaskViewHolder extends RecyclerView.ViewHolder { 5 usages
    TextView taskName, taskDescription, taskTime, taskLabel; 6 usages
    ImageButton editButton, deleteButton; 2 usages
    CheckBox taskCheckBox; 3 usages
    View taskItemContainer; 3 usages

    public TaskViewHolder(@NonNull View itemView) { 1 usage
        super(itemView);
        taskName = itemView.findViewById(R.id.taskName);
        taskDescription = itemView.findViewById(R.id.taskDescription);
        taskTime = itemView.findViewById(R.id.taskTime);
        taskLabel = itemView.findViewById(R.id.taskLabel);
        editButton = itemView.findViewById(R.id.editButton);
        deleteButton = itemView.findViewById(R.id.deleteButton);
        taskCheckBox = itemView.findViewById(R.id.taskCheckBox);
        taskItemContainer = itemView.findViewById(R.id.taskItemContainer);
    }
}
```

Figure 32. Others function in `TaskAdapter`.

In `TaskAdapter.java`, the `updateTaskUI()` method adjusts the visual style of each task item based on its completion status. If a task is marked as completed, it applies a strikethrough to the task name and description and changes the background color. Otherwise, it removes the strikethrough and resets the background. The `getItemCount()` method returns the total number of tasks in the list. The `TaskViewHolder` class initializes UI elements, such as task name, description, time, label, edit, delete buttons, checkbox, and container view for each task item, allowing easy access and modification within the adapter.

2.2.6 strings.xml & styles.xml:

```
<resources>
    <string name="app_name">Logbook2</string>
    <string-array name="filter_options">
        <item>All</item>
        <item>Completed</item>
        <item>Not Completed</item>
    </string-array>
    <string-array name="label_options">
        <item>Work</item>
        <item>Personal</item>
        <item>Team</item>
    </string-array>
</resources>
```

Figure 33. strings.xml

```
<resources>
    <style name="CustomTimePicker" parent="ThemeOverlay.MaterialComponents.TimePicker">
        <item name="colorAccent">@color/colorAccent</item>
        <item name="android:textColorPrimary">@color/colorPrimary</item>
        <item name="android:textColorSecondary">@color/colorPrimaryDark</item>
    </style>
</resources>
```

Figure 34. styles.xml

The `strings.xml` file defines text resources for the app, including task filter and label options, ensuring consistent values across the UI. The `styles.xml` file customizes UI components like `CustomTimePicker` with specific color themes, maintaining a cohesive look and feel.

III. Logbook 3:

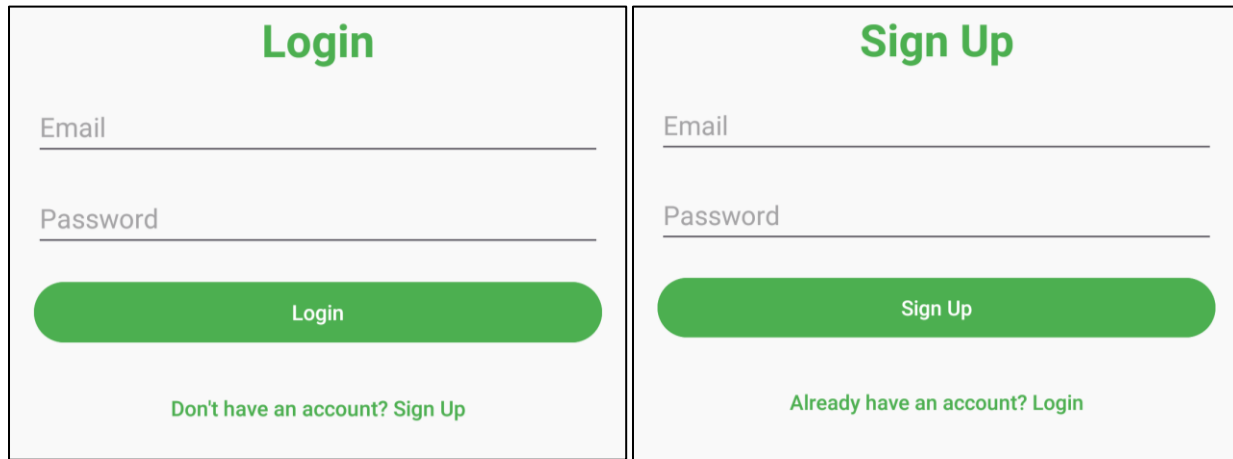
1. Basic Information

1.1 Student name	Lieu Vinh Khoi
1.2 Who did you work with? Note that for logbook exercises you are allowed to work with one other person as long as you give their name and login id and both contribute to the work.	Name: Login id:
1.3 Which Exercise is this? Tick as appropriate.	• Exercise 1 ☐ • Exercise 2 ☐ • Exercise 3 ☒
1.4 How well did you complete the exercise? Tick as appropriate.	• I tried but couldn't complete it ☐ • I did it but I feel I should have done better ☐ • I did everything that was asked ☐ • I did more than was asked for ☒
1.5 Briefly explain your answer to question 1.4 Without any explanation/justification, your scores will be deducted.	I developed a task management application similar to Logbook 2, where users can add, edit, delete, and categorize tasks, as well as mark them as completed. However, Logbook 3 adds user registration and login, ensuring personalized access. Additionally, tasks are stored in SQLite, so task data and completion status are preserved across sessions, making the application more secure and resilient than before.

2. Exercise answer

2.1 Screenshots demonstrating what you achieved

2.1.1 Application Interfaces



Login

Email

Password

Login

Don't have an account? Sign Up

Sign Up

Email

Password

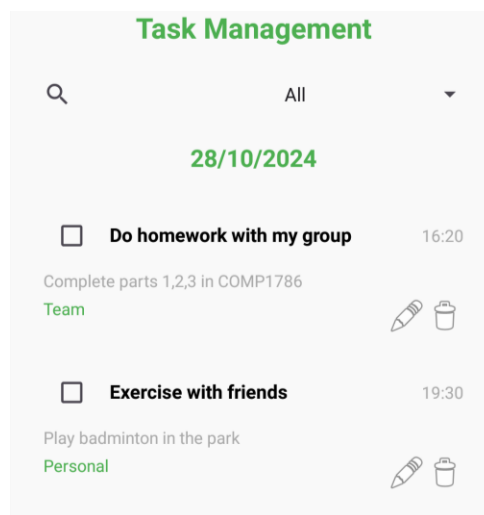
Sign Up

Already have an account? Login

Figure 35. Login and Signup Interface

Logbook 3 retains the familiar interface of Logbook 2, with a toolbar featuring search and filter options, a date header for displaying tasks by date, and a RecyclerView that highlights completed tasks. Each task has options to edit, delete, or mark as completed, and a floating action button allows adding new tasks. With SQLite integration, task details and completion status are saved across sessions. Additionally, Logbook 3 introduces secure login and signup screens, allowing users to create accounts with hashed passwords and log in to access their personalized tasks, enhancing both usability and security.

2.1.2 SQLite Integration



Task Management

🔍 All ▼

28/10/2024

☐ **Do homework with my group** 16:20

Complete parts 1,2,3 in COMP1786
Team  

☐ **Exercise with friends** 19:30

Play badminton in the park
Personal  

Figure 36. Main interface

	id	name	description	time	label	date	completed
1	2	Do homework with my group	Complete parts 1,2,3 in COMP1786	18:20	Team	28/10/2024	0
2	3	Exercise with friends	Play badminton in the park	19:30	Personal	28/10/2024	0

Figure 37. Task table in Database

	id	email	password
1	1	khoivl.net@gmail.com	d28a96c88d5cc79b942b5bbad871241959c8ef0f4093393231df2272087ad4ec

Figure 38. User table in Database

Logbook 3 utilizes SQLite to store tasks and user data persistently. The tasks table keeps task details like name, description, time, label, date, and completion status, enabling consistent task management. The users table securely stores user emails and hashed passwords, supporting individual accounts and data security across sessions.

2.1.3 Application features

2.1.3.1 Login Interface

Login

Email

Password

Login

Don't have an account? Sign Up

Figure 39. Login interface

Figure 39 shows the login interface of the application, which includes fields for entering email and password. Users can log in by entering their credentials and pressing the "Login" button. If they don't have an account, they can tap "Sign Up" to navigate to the registration screen. This clean and intuitive layout makes it easy for users to log in or proceed to the sign-up page.

2.1.3.2 Sign Up Interface

The image shows a 'Sign Up' interface. At the top, the text 'Sign Up' is displayed in a large, bold, green font. Below this, there are two input fields: 'Email' and 'Password', each with a horizontal line underneath. A large, rounded green button with the text 'Sign Up' in white is centered below the input fields. At the bottom, there is a link that says 'Already have an account? Login' in a smaller green font.

Figure 40. Signup interface

Figure 40 shows the sign-up interface, which includes fields for users to enter their email and password. By clicking the "Sign Up" button, users can create a new account. If they already have an account, they can tap "Login" at the bottom to return to the login screen. This interface is designed for easy and straightforward user registration.

2.1.4 Code that you wrote

2.1.4.1 Directory structure

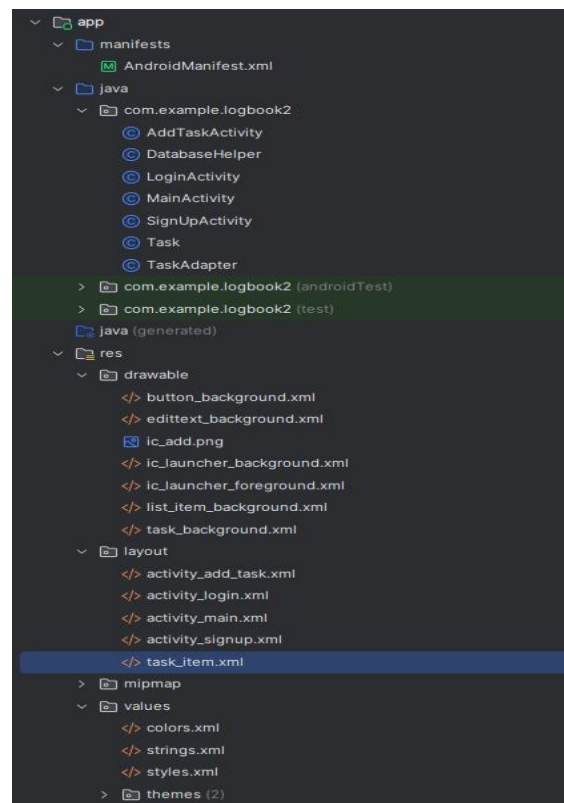


Figure 41. Directory structure of Logbook 3

The Logbook 3 folder structure builds on Logbook 2's interface, with key additions: `LoginActivity` and `SignUpActivity` for user authentication, and `DatabaseHelper` for managing SQLite operations to store user and task data. These added files enable secure login functionality and persistent task storage. Other familiar components, like drawable for icons, layout for screen designs, and values for UI consistency, remain unchanged from Logbook 2.

2.1.4.2 SignUpActivity.java:

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_signup);

    etEmail = findViewById(R.id.etEmail);
    etPassword = findViewById(R.id.etPassword);
    btnSignUp = findViewById(R.id.btnSignUp);
    btnGoToLogin = findViewById(R.id.btnGoToLogin);

    dbHelper = new DatabaseHelper( context: this);

    btnSignUp.setOnClickListener(v -> {
        String email = etEmail.getText().toString().trim();
        String password = etPassword.getText().toString().trim();

        if (TextUtils.isEmpty(email) || TextUtils.isEmpty(password)) {
            Toast.makeText( context: SignUpActivity.this, text: "Please fill all fields", Toast.LENGTH_SHORT).show();
            return;
        }

        if (dbHelper.checkEmailExists(email)) {
            Toast.makeText( context: SignUpActivity.this, text: "Email already exists. Please use a different email.", Toast.LENGTH_SHORT).show();
            return;
        }

        String hashedPassword = hashPassword(password);

        if (dbHelper.addUser(email, hashedPassword)) {
            Toast.makeText( context: SignUpActivity.this, text: "Sign Up Successful", Toast.LENGTH_SHORT).show();
            startActivity(new Intent( packageContext: SignUpActivity.this, LoginActivity.class));
            finish();
        } else {
            Toast.makeText( context: SignUpActivity.this, text: "Failed to sign up", Toast.LENGTH_SHORT).show();
        }
    });

    btnGoToLogin.setOnClickListener(v -> {
        startActivity(new Intent( packageContext: SignUpActivity.this, LoginActivity.class));
    });
}
```

Figure 42. onCreate() function in SignUpActivity

The `onCreate()` function in `SignUpActivity` sets up the signup screen by linking UI elements (like email and password input fields) and configuring button actions. When the "Sign Up" button is clicked, it validates that the fields aren't empty and checks if the email already exists in the database. If the input is valid, the password is hashed for security, and the new user is added to

the database. A success message is shown, and the user is redirected to the login screen. If signup fails or the email already exists, appropriate error messages are displayed.

```
// Method to hash password using SHA-256
private String hashPassword(String password) { 1 usage
    try {
        MessageDigest digest = MessageDigest.getInstance("SHA-256");
        byte[] hash = digest.digest(password.getBytes());
        StringBuilder hexString = new StringBuilder();
        for (byte b : hash) {
            String hex = Integer.toHexString(b & 0xff);
            if (hex.length() == 1) hexString.append('0');
            hexString.append(hex);
        }
        return hexString.toString();
    } catch (NoSuchAlgorithmException e) {
        e.printStackTrace();
        return null;
    }
}
```

Figure 43. hashPassword() function in SignUpActivity

The `hashPassword()` function in `SignUpActivity` securely hashes the user's password using the SHA-256 algorithm. It converts the password into a byte array, applies SHA-256 to produce a hash, and then formats the hash into a hexadecimal string for storage. This process ensures that passwords are stored securely by protecting them against direct exposure in the database.

2.1.4.3 LoginActivity.java:

```

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_login);

    etEmail = findViewById(R.id.etEmail);
    etPassword = findViewById(R.id.etPassword);
    btnLogin = findViewById(R.id.btnLogin);
    btnGoToSignUp = findViewById(R.id.btnGoToSignUp);

    dbHelper = new DatabaseHelper(context, this);

    btnLogin.setOnClickListener(v -> {
        String email = etEmail.getText().toString().trim();
        String password = etPassword.getText().toString().trim();

        if (TextUtils.isEmpty(email) || TextUtils.isEmpty(password)) {
            Toast.makeText(context, LoginActivity.this, text: "Please fill all fields", Toast.LENGTH_SHORT).show();
            return;
        }

        String hashedPassword = hashPassword(password);

        if (dbHelper.checkUser(email, hashedPassword)) {
            Toast.makeText(context, LoginActivity.this, text: "Login Successful", Toast.LENGTH_SHORT).show();
            startActivity(new Intent(packageContext, LoginActivity.this, MainActivity.class));
            finish();
        } else {
            Toast.makeText(context, LoginActivity.this, text: "Invalid credentials", Toast.LENGTH_SHORT).show();
        }
    });

    btnGoToSignUp.setOnClickListener(v -> {
        startActivity(new Intent(packageContext, LoginActivity.this, SignUpActivity.class));
    });
}

```

Figure 44. onCreate() function in LoginActivity

The `onCreate()` function in `LoginActivity` includes a step where the entered password is hashed using the same `hashPassword()` method that is used in `SignUpActivity`. This ensures that user credentials are stored securely in the database by converting passwords into SHA-256 hashed strings rather than storing them as plain text. When a user attempts to log in, `LoginActivity` hashes the entered password and then compares it to the stored hashed password in SQLite. This approach provides a consistent and secure authentication method across both signup and login processes, enhancing the security of user data.

2.1.4.4 DatabaseHelper.java:

```
public class DatabaseHelper extends SQLiteOpenHelper { 9 usages
    private static final String DATABASE_NAME = "tasks.db"; 1 usage
    private static final int DATABASE_VERSION = 1; 1 usage
    private static final String TABLE_TASKS = "tasks"; 7 usages
    private static final String COLUMN_TASK_ID = "id"; 5 usages
    private static final String COLUMN_TASK_NAME = "name"; 4 usages
    private static final String COLUMN_TASK_DESCRIPTION = "description"; 4 usages
    private static final String COLUMN_TASK_TIME = "time"; 4 usages
    private static final String COLUMN_TASK_LABEL = "label"; 4 usages
    private static final String COLUMN_TASK_DATE = "date"; 4 usages
    private static final String COLUMN_TASK_COMPLETED = "completed"; 5 usages
    private static final String TABLE_USERS = "users"; 5 usages
    private static final String COLUMN_USER_ID = "id"; 1 usage
    private static final String COLUMN_USER_EMAIL = "email"; 4 usages
    private static final String COLUMN_USER_PASSWORD = "password"; 3 usages
    public DatabaseHelper(Context context) { 4 usages
        super(context, DATABASE_NAME, factory: null, DATABASE_VERSION);
    }

    @Override
    public void onCreate(SQLiteDatabase db) {
        String createTaskTable = "CREATE TABLE " + TABLE_TASKS + " (" +
            COLUMN_TASK_ID + " INTEGER PRIMARY KEY AUTOINCREMENT, " +
            COLUMN_TASK_NAME + " TEXT, " +
            COLUMN_TASK_DESCRIPTION + " TEXT, " +
            COLUMN_TASK_TIME + " TEXT, " +
            COLUMN_TASK_LABEL + " TEXT, " +
            COLUMN_TASK_DATE + " TEXT, " +
            COLUMN_TASK_COMPLETED + " INTEGER)";
        db.execSQL(createTaskTable);

        String createUserTable = "CREATE TABLE " + TABLE_USERS + " (" +
            COLUMN_USER_ID + " INTEGER PRIMARY KEY AUTOINCREMENT, " +
            COLUMN_USER_EMAIL + " TEXT UNIQUE, " +
            COLUMN_USER_PASSWORD + " TEXT)";
        db.execSQL(createUserTable);
    }

    @Override no usages
    public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
        db.execSQL("DROP TABLE IF EXISTS " + TABLE_TASKS);
        db.execSQL("DROP TABLE IF EXISTS " + TABLE_USERS);
        onCreate(db);
    }
}
```

Figure 45. DatabaseHelper (Part 1)

The DatabaseHelper class manages SQLite database creation for the the app. It defines two tables: `tasks` for storing task details (including name, description, time, label, date, and completion status) and `users` for storing user credentials (email and hashed password). In the `onCreate` method, SQL statements create the `tasks` and `users` tables if they don't already

exist. Each table has unique columns and uses `INTEGER PRIMARY KEY AUTOINCREMENT` for the `id` field to uniquely identify entries. The `onUpgrade` method handles database upgrades by dropping existing tables and recreating them, ensuring the schema stays updated. This setup supports task management and secure user authentication.

```
public void addTask(Task task) { 1 usage
    SQLiteDatabase db = this.getWritableDatabase();
    ContentValues values = new ContentValues();
    values.put(COLUMN_TASK_NAME, task.getName());
    values.put(COLUMN_TASK_DESCRIPTION, task.getDescription());
    values.put(COLUMN_TASK_TIME, task.getTime());
    values.put(COLUMN_TASK_LABEL, task.getLabel());
    values.put(COLUMN_TASK_DATE, task.getDate());
    values.put(COLUMN_TASK_COMPLETED, task.isCompleted() ? 1 : 0);
    db.insert(TABLE_TASKS, nullColumnHack: null, values);
    db.close();
}

public ArrayList<Task> getAllTasks() { 1 usage
    ArrayList<Task> taskList = new ArrayList<>();
    SQLiteDatabase db = this.getReadableDatabase();
    Cursor cursor = db.rawQuery( sql: "SELECT * FROM " + TABLE_TASKS, selectionArgs: null);

    if (cursor.moveToFirst()) {
        do {
            int id = cursor.getInt(cursor.getColumnIndexOrThrow(COLUMN_TASK_ID));
            String name = cursor.getString(cursor.getColumnIndexOrThrow(COLUMN_TASK_NAME));
            String description = cursor.getString(cursor.getColumnIndexOrThrow(COLUMN_TASK_DESCRIPTION));
            String time = cursor.getString(cursor.getColumnIndexOrThrow(COLUMN_TASK_TIME));
            String label = cursor.getString(cursor.getColumnIndexOrThrow(COLUMN_TASK_LABEL));
            String date = cursor.getString(cursor.getColumnIndexOrThrow(COLUMN_TASK_DATE));
            boolean completed = cursor.getInt(cursor.getColumnIndexOrThrow(COLUMN_TASK_COMPLETED)) == 1;
            Task task = new Task(id, name, description, time, label, date, completed);
            taskList.add(task);
        } while (cursor.moveToNext());
    }
    cursor.close();
    db.close();
    return taskList;
}
```

Figure 46. DatabaseHelper (Part 2)

In this part of `DatabaseHelper`, the `addTask` method saves a task to the `tasks` table in the SQLite database by inserting values like task name, description, time, label, date, and completion status. The `getAllTasks` method retrieves all tasks from the database, constructs `Task` objects for each entry, and stores them in an `ArrayList`. This list is returned for display or processing in

the app, allowing persistent task storage and retrieval. These methods enable seamless saving and loading of tasks for users.

```
public void updateTask(Task task) { 1 usage
    SQLiteDatabase db = this.getWritableDatabase();
    ContentValues values = new ContentValues();
    values.put(COLUMN_TASK_NAME, task.getName());
    values.put(COLUMN_TASK_DESCRIPTION, task.getDescription());
    values.put(COLUMN_TASK_TIME, task.getTime());
    values.put(COLUMN_TASK_LABEL, task.getLabel());
    values.put(COLUMN_TASK_DATE, task.getDate());
    values.put(COLUMN_TASK_COMPLETED, task.isCompleted() ? 1 : 0);
    db.update(TABLE_TASKS, values, whereClause: COLUMN_TASK_ID + "=?", new String[]{String.valueOf(task.getId())});
    db.close();
}

public void updateTaskCompletedStatus(int taskId, boolean isCompleted) { 1 usage
    SQLiteDatabase db = this.getWritableDatabase();
    ContentValues values = new ContentValues();
    values.put(COLUMN_TASK_COMPLETED, isCompleted ? 1 : 0);
    db.update(TABLE_TASKS, values, whereClause: COLUMN_TASK_ID + "=?", new String[]{String.valueOf(taskId)});
    db.close();
}

public void deleteTask(int taskId) { 1 usage
    SQLiteDatabase db = this.getWritableDatabase();
    db.delete(TABLE_TASKS, whereClause: COLUMN_TASK_ID + "=?", new String[]{String.valueOf(taskId)});
    db.close();
}
```

Figure 47. DatabaseHelper (Part 3)

In this section of `DatabaseHelper`, the `updateTask` method updates an existing task in the `tasks` table with new details like name, description, time, label, date, and completion status. The `updateTaskCompletedStatus` method specifically updates the task's completion status based on the task ID. The `deleteTask` method removes a task from the database by its ID. These methods enable modifying and deleting tasks in the SQLite database, ensuring data reflects any user changes.

```

public boolean addUser(String email, String hashedPassword) { 1 usage
    SQLiteDatabase db = this.getWritableDatabase();
    ContentValues values = new ContentValues();
    values.put(COLUMN_USER_EMAIL, email);
    values.put(COLUMN_USER_PASSWORD, hashedPassword); // Store hashed password

    long result = db.insert(TABLE_USERS, nullColumnHack, null, values);
    db.close();
    return result != -1;
}

public boolean checkUser(String email, String hashedPassword) { 1 usage
    SQLiteDatabase db = this.getReadableDatabase();
    Cursor cursor = db.rawQuery("SELECT * FROM " + TABLE_USERS + " WHERE " + COLUMN_USER_EMAIL + "=? AND " + COLUMN_USER_PASSWORD + "=?",
        new String[]{email, hashedPassword});
    boolean exists = cursor.moveToFirst();
    cursor.close();
    db.close();
    return exists;
}

public boolean checkEmailExists(String email) { 1 usage
    SQLiteDatabase db = this.getReadableDatabase();
    Cursor cursor = db.rawQuery("SELECT * FROM " + TABLE_USERS + " WHERE " + COLUMN_USER_EMAIL + "=?", new String[]{email});
    boolean exists = cursor.moveToFirst();
    cursor.close();
    db.close();
    return exists;
}

```

Figure 48. DatabaseHelper (Part 4)

This part of `DatabaseHelper` includes methods for user management. The `addUser` method adds a new user with an email and hashed password to the `users` table, ensuring data security. The `checkUser` method verifies if a user exists with the provided email and hashed password, enabling secure login validation. The `checkEmailExists` method checks if an email is already registered, which helps prevent duplicate accounts during signup. These methods support secure user authentication and registration functionalities in the application.

2.1.4.5 MainActivity.java:

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);

    RecyclerView taskRecyclerView = findViewById(R.id.taskRecyclerView);
    FloatingActionButton addTaskButton = findViewById(R.id.addTaskButton);
    dateHeader = findViewById(R.id.dateHeader);
    emptyTextView = findViewById(R.id.emptyTextView);
    SearchView searchView = findViewById(R.id.searchView);
    Spinner filterSpinner = findViewById(R.id.filterSpinner);

    dbHelper = new DatabaseHelper(context, this); // Initialize DatabaseHelper

    taskList = new ArrayList<>();
    allTasks = dbHelper.getAllTasks(); // Load tasks from the database
    taskAdapter = new TaskAdapter(taskList, context, this, this::deleteTask);

    taskRecyclerView.setLayoutManager(new LinearLayoutManager(context, this));
    taskRecyclerView.setAdapter(taskAdapter);

    Calendar calendar = Calendar.getInstance();
    SimpleDateFormat dateFormat = new SimpleDateFormat("dd/MM/yyyy", Locale.getDefault());
    selectedDate = dateFormat.format(calendar.getTime());
    dateHeader.setText(selectedDate);
    filterTasks();

    addTaskButton.setOnClickListener(v -> {
        Intent intent = new Intent(packageContext, MainActivity.this, AddTaskActivity.class);
        startActivityForResult(intent, REQUEST_CODE_ADD_TASK);
    });

    dateHeader.setOnClickListener(v -> showDatePickerDialog());
}
```

Figure 49. onCreate() function in MainActivity

In `MainActivity.java` of Logbook 3, the main change from Logbook 2 is the addition of SQLite functionality. This includes initializing `DatabaseHelper` and using it to load tasks from the database into `allTasks`, making tasks persist across sessions. The `dbHelper.getAllTasks()` call fetches stored tasks from SQLite, which are then managed and displayed within the activity. This ensures that task data, including completion status, is saved and available even after closing the app, enhancing the application's functionality and user experience.

```

private void deleteTask(int position) { 1 usage
    Task task = taskList.get(position);
    dbHelper.deleteTask(task.getId()); // Delete task from the database
    allTasks.remove(task);
    taskList.remove(position);
    taskAdapter.notifyItemRemoved(position);
    taskAdapter.notifyItemRangeChanged(position, taskList.size());

    if (taskList.isEmpty()) {
        emptyTextView.setVisibility(View.VISIBLE);
    }
}

@Override
protected void onActivityResult(int requestCode, int resultCode, @Nullable Intent data) {
    super.onActivityResult(requestCode, resultCode, data);

    if ((requestCode == REQUEST_CODE_ADD_TASK || requestCode == REQUEST_CODE_EDIT_TASK) && resultCode == RESULT_OK && data != null) {
        String taskName = data.getStringExtra("taskName");
        String taskDescription = data.getStringExtra("taskDescription");
        String taskTime = data.getStringExtra("taskTime");
        String taskLabel = data.getStringExtra("taskLabel");
        String taskDate = data.getStringExtra("taskDate");
        boolean isEditMode = data.hasExtra("taskPosition");

        if (isEditMode) {
            int position = data.getIntExtra("taskPosition", -1);
            Task task = allTasks.get(position);
            task.setName(taskName);
            task.setDescription(taskDescription);
            task.setTime(taskTime);
            task.setLabel(taskLabel);
            task.setDate(taskDate);
            dbHelper.updateTask(task);
            taskAdapter.notifyItemChanged(position);
        } else {
            Task newTask = new Task(taskName, taskDescription, taskTime, taskLabel, taskDate);
            dbHelper.addTask(newTask);
            allTasks.add(newTask);
        }
        filterTasks();
    }
}

```

Figure 50. Others function in MainActivity

In the code snippet from figure 50, `MainActivity.java` in Logbook 3 incorporates SQLite to manage task persistence. Unlike Logbook 2, this version utilizes `dbHelper` methods to add, update, and delete tasks directly in the database (`addTask`, `updateTask`, `deleteTask`). This ensures that all task changes, including edits and deletions, are saved in SQLite, allowing task data to persist between app sessions and providing a more reliable task management experience.

2.1.4.6 Task.java:

```
public class Task { 15 usages
    private int id; 3 usages
    private String name; 4 usages
    private String description; 4 usages
    private String time; 4 usages
    private String label; 4 usages
    private String date; 4 usages
    private boolean completed; 4 usages

    public Task(int id, String name, String description, String time, String label, String date, boolean completed) { 15 usages
        this.id = id;
        this.name = name;
        this.description = description;
        this.time = time;
        this.label = label;
        this.date = date;
        this.completed = completed;
    }

    public Task(String name, String description, String time, String label, String date) { 15 usages
        this.name = name;
        this.description = description;
        this.time = time;
        this.label = label;
        this.date = date;
        this.completed = false;
    }

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getDescription() { return description; } 4 usages
    public void setDescription(String description) { this.description = description; } 1 usage
    public String getTime() { return time; } 4 usages
    public void setTime(String time) { this.time = time; } 1 usage
    public String getLabel() { return label; } 4 usages
    public void setLabel(String label) { this.label = label; } 1 usage
    public String getDate() { return date; } 4 usages
    public void setDate(String date) { this.date = date; } 1 usage
    public boolean isCompleted() { return completed; } 6 usages
    public void setCompleted(boolean completed) { this.completed = completed; } 1 usage
}
```

Figure 51. Task class

In this `Task` class for Logbook 3, modifications were made to support SQLite integration. A new `id` field was added to uniquely identify each task in the database. Additionally, constructors and getter/setter methods now handle this `id` along with the `completed` status. This setup allows each task's details, including its completion state, to be stored and retrieved from the SQLite database, enabling persistence across app sessions.

2.1.4.7 TaskAdapter.java:

```
@Override
public void onBindViewHolder(@NonNull TaskViewHolder holder, int position) {
    Task task = taskList.get(position);

    holder.taskName.setText(task.getName());
    holder.taskDescription.setText(task.getDescription());
    holder.taskTime.setText(task.getTime());
    holder.taskLabel.setText(task.getLabel());
    holder.taskCheckBox.setChecked(task.isCompleted());

    updateTaskUI(holder, task.isCompleted());

    holder.taskCheckBox.setOnCheckedChangeListener((buttonView, isChecked) -> {
        task.setCompleted(isChecked);
        updateTaskUI(holder, isChecked);

        DatabaseHelper dbHelper = new DatabaseHelper(context);
        dbHelper.updateTaskCompletedStatus(task.getId(), isChecked);
    });

    holder.editButton.setOnClickListener(v -> {
        Intent intent = new Intent(context, AddTaskActivity.class);
        intent.putExtra("isEditMode", true);
        intent.putExtra("taskPosition", position);
        intent.putExtra("taskName", task.getName());
        intent.putExtra("taskDescription", task.getDescription());
        intent.putExtra("taskTime", task.getTime());
        intent.putExtra("taskLabel", task.getLabel());
        intent.putExtra("taskDate", task.getDate());
        ((MainActivity) context).startActivityForResult(intent, MainActivity.REQUEST_CODE_EDIT_TASK);
    });

    holder.deleteButton.setOnClickListener(v -> new AlertDialog.Builder(context)
        .setTitle("Confirm Delete")
        .setMessage("Are you sure you want to delete this task?")
        .setPositiveButton("Yes", (dialog, which) -> deleteTaskCallback.deleteTask(position))
        .setNegativeButton("No", listener: null)
        .show());
}
```

Figure 52. `onBindViewHolder()` function in `TaskAdapter`

In the `onBindViewHolder()` function in Logbook 3's `TaskAdapter` when a task's checkbox is toggled, the `DatabaseHelper` is used to update the task's completion status in the SQLite database with the `updateTaskCompletedStatus` method. This ensures that changes to the task's completion state persist across app sessions, unlike in Logbook 2, where such persistence was not implemented.